Stock Price Trend Prediction with LSTM

Project completion report | Ekundayo Monday Edeki | github.com/edeki101/stock-price-lstm

Introduction

Share prices are noisy time series, and forecasting them from past behavior is a standard test case for sequence models. This project builds an end-to-end pipeline that fetches daily price history, engineers trend indicators, trains a Long Short-Term Memory (LSTM) network in Keras to predict the next trading day's closing price, and presents the results in a Streamlit dashboard. The goal is a correct, reproducible workflow that is evaluated honestly against a simple baseline.

Abstract

A two-layer LSTM was trained on 60-day windows of scale-free features (log return, distance of price from its 20- and 50-day moving averages, and RSI-14) to predict the next-day log return, from which the price forecast is rebuilt. The pipeline was run on 2,160 daily Apple (AAPL) observations from 2000-03-14 to 2008-10-14 (public Yahoo-format dataset), split chronologically 80/20. On the 432-day held-out period the model reached an RMSE of 4.14 USD, MAE 2.99 and MAPE 2.10%, with 55.6% directional accuracy. These figures are essentially identical to the naive baseline that predicts tomorrow's price as today's price (RMSE 4.14), so the model tracks the price closely but shows no demonstrable predictive edge.

Tools Used

Python 3 (pipeline); **yfinance** (Yahoo Finance data API); **Pandas / NumPy** (cleaning, indicators, sequences); **scikit-learn** (standardization); **Keras on TensorFlow** (LSTM model, training callbacks, saved weights); **Matplotlib** (graphs); **Jupyter** (notebook); **Streamlit** (dashboard); **Git/GitHub** (version control and delivery).

Steps Involved in Building the Project

- **Fetch data.** Daily OHLCV prices are pulled with the yfinance API (split/dividend-adjusted). In this run Yahoo Finance was unreachable, so the notebook's built-in fallback loaded a public Yahoo-format AAPL dataset (2000–2008); on a working connection it pulls live data.
- **Indicators.** 20- and 50-day moving averages and Wilder's 14-day RSI are computed from past data only, avoiding look-ahead leakage.
- **Normalize and prepare.** Inputs are standardized with scalers fit on the training period only. Overlapping 60-day windows become samples; the target is the next-day log return.
- **Build the LSTM.** LSTM(64) → Dropout(0.2) → LSTM(32) → Dropout(0.2) → Dense(16, ReLU) → Dense(1); Adam optimizer, MSE loss.
- **Train and validate.** Chronological split, no shuffling; the last 10% of training windows form the validation set; early stopping and learning-rate reduction (14 epochs ran).
- **Plot predictions vs actual.** Predicted prices (previous close × exp(predicted return)) are overlaid on actual prices, with loss curves and price/MA/RSI charts.
- **Dashboard.** A Streamlit app lets the user choose a ticker and window, trains the model, and shows forecasts, metrics against the baseline and the indicator charts.

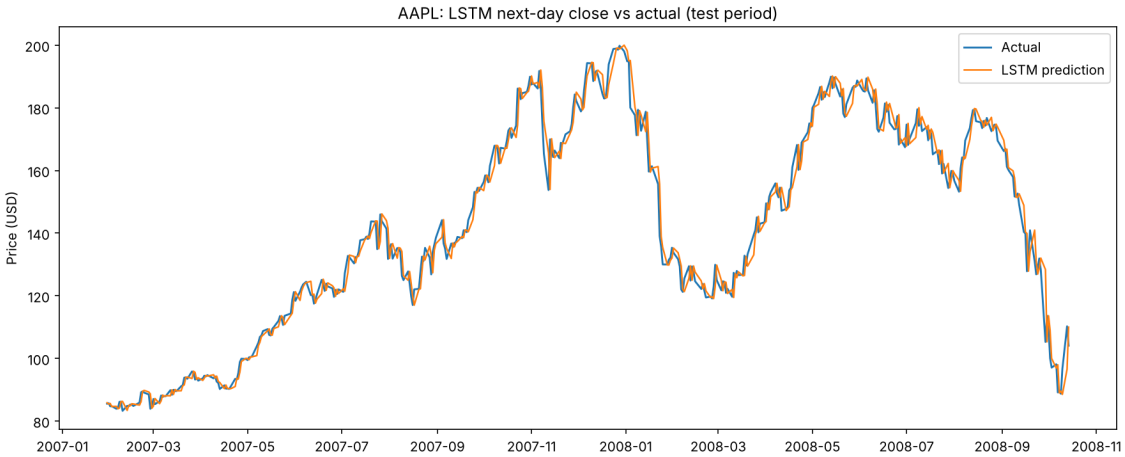


Figure 1. LSTM next-day close vs actual on the held-out test period (AAPL, public 2000–2008 dataset).

Model	RMSE	MAE	MAPE	Direction
LSTM	4.140	2.990	2.10%	55.6%
Naive (tomorrow = today)	4.141	3.001	2.11%	n/a

Table 1. Test-period metrics. Share of up-days in the test period: 53.7%.

Supporting graphs

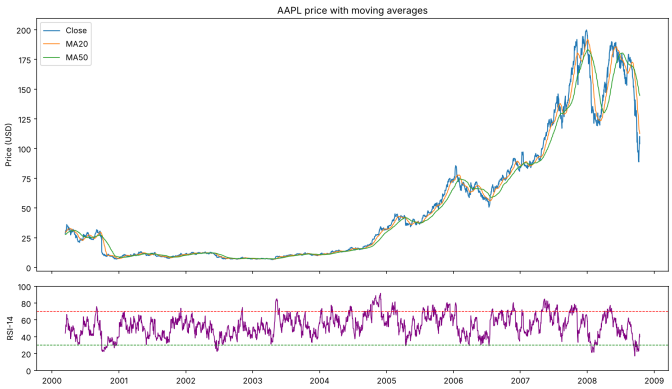




Figure 2. Left: price with 20/50-day moving averages and RSI-14. Right: training vs validation loss. The nearly flat loss curve shows the network learned little beyond the average return, consistent with the baseline-level accuracy.

Conclusion

The project delivers a complete, leak-free LSTM forecasting workflow: live data ingestion, indicator engineering, chronological validation, saved model weights, graphs and a deployable dashboard. A first version that predicted raw price levels failed once prices rose beyond the training range (RMSE 43.2 vs 4.14 for the baseline); switching to returns and scale-free features fixed this. The final model matches, but does not beat, a naive baseline, and its directional accuracy is only about two points above the share of up-days on 432 samples, which is not statistically meaningful. Markets are close to a random walk, so this is the expected outcome for price-only inputs. Next steps: add volume, volatility and market-wide features, test walk-forward validation across several tickers, and evaluate with trading-cost-aware metrics. Educational work, not investment advice.